Chapter 2

Instructions: Language of the Computer

2.7 Compiling Loop

- C code:

```
while (save[i] == k) i += 1;
```

  - i in x22, k in x24, address of save in x25
- Compiled RISC-V code:

```
Loop: slli x10, x22, 3
      add  x10, x10, x25
      ld   x9, 0(x10)
      bne  x9, x24, Exit
      addi x22, x22, 1
      beq  x0, x0, Loop
Exit: …
```

2.7 More Conditional Oprs.

- `blt rs1, rs2, L1`
  - if (rs1 < rs2) branch to instruction labeled L1
- `bge rs1, rs2, L1`
  - if (rs1 >= rs2) branch to instruction labeled L1
- Example
  - if (a > b) a += 1;
  - a in x22, b in x23

```
bge  x23, x22, Exit      // branch if b >= a
addi x22, x22, 1
Exit:
```

2.7 Signed vs. Unsigned

- Signed comparison: blt, bge
- Unsigned comparison: bltu, bgeu
- Example
  - x22 = 1111 1111 1111 1111 1111 1111 1111 1111
  - x23 = 0000 0000 0000 0000 0000 0000 0000 0001
  - `x22 < x23 // signed`
    - –1 < +1
  - `x22 > x23 // unsigned`
    - +4,294,967,295 > +1

2.8 Procedure Calling

- Steps required
  1. Place parameters in registers x10 to x17
  2. Transfer control to procedure
  3. Acquire storage for procedure
  4. Perform procedure's operations
  5. Place result in register for caller
  6. Return to place of call (address in x1)

2.8 Procedure Call Instr.

- Procedure call: jump and link

  `jal x1, ProcedureLabel`

  - Address of following instruction put in x1
  - Jumps to target address
- Procedure return: jump and link register

  `jalr x0, 0(x1)`

  - Like jal, but jumps to 0 + address in x1
  - Use x0 as rd (x0 cannot be changed)
  - Can also be used for computed jumps
    - e.g., for case/switch statements

2.8 Leaf Procedure Example

- C code:

```
long long int leaf_example (
      long long int g, long long int h,
      long long int i, long long int j) {
  long long int f;
  f = (g + h) - (i + j);
  return f;
}
```

  - Arguments g, …, j in x10, …, x13
  - f in x20
  - temporaries x5, x6
  - Need to save x5, x6, x20 on stack

2.8 Leaf Procedure Example

- RISC-V code:

```
leaf_example:
   addi sp,sp,-24
   sd   x5,16(sp)
   sd   x6,8(sp)
   sd   x20,0(sp
   add  x5,x10,x11
   add  x6,x12,x1
   sub  x20,x5,x6
   addi x10,x20,0
   ld   x20,0(sp)
   ld   x6,8(sp)
   ld   x5,16(sp)
   addi sp,sp,24
   jalr x0,0(x1)
```

Save x5, x6, x20 on stack

x5 = g + h

x6 = i + j

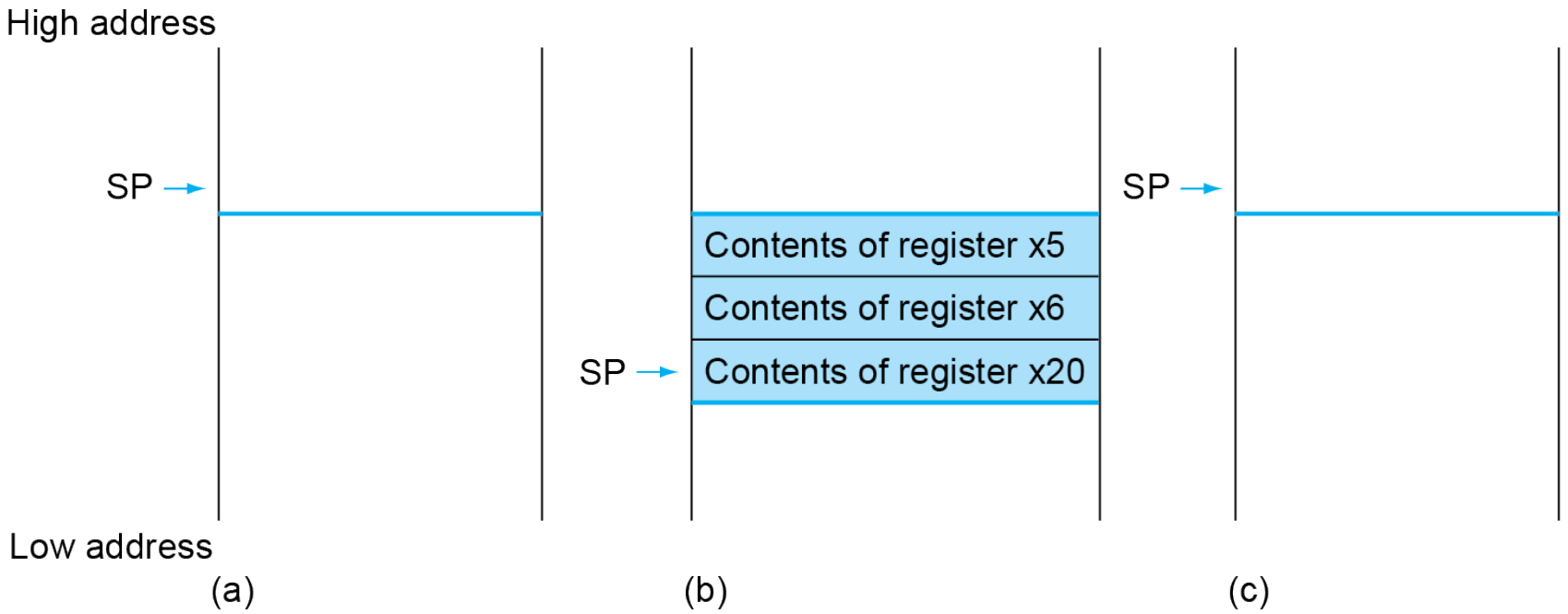
f = x5 – x6

copy f to return register

Resore x5, x6, x20 from stack

Return to caller

2.8 Local Data on the Stack



2.8 Register Usage

- x5 – x7, x28 – x31: temporary registers
  - Not preserved by the callee
- x8 – x9, x18 – x27: saved registers
  - If used, the callee saves and restores them

2.8 Non-Leaf Procedures

- Procedures that call other procedures
- For nested call, caller needs to save on the stack:
  - Its return address
  - Any arguments and temporaries needed after the call
- Restore from the stack after the call

2.8 Non-Leaf Procedure Ex.

- C code:

```
long long int fact (long long int n)
{
  if (n < 1) return f;
  else return n * fact(n - 1);
}
```

  - Argument n in x10
  - Result in x10

2.8 Leaf Procedure Example

- RISC-V code:

```
fact:
    addi sp,sp,-16
    sd   x1,8(sp)
    sd   x10,0(sp)
    addi x5,x10,-1
    bge  x5,x0,L1
    addi x10,x0,1
    addi sp,sp,16
    jalr x0,0(x1)
L1: addi x10,x10,-1
    jal  x1,fact
    addi x6,x10,0
    ld   x10,0(sp)
    ld   x1,8(sp)
    addi sp,sp,16
    mul  x10,x10,x6
    jalr x0,0(x1)
```

Save return address and n on stack

x5 = n - 1

if n >= 1, go to L1

Else, set return value to 1

Pop stack, don't bother restoring values

Return

n = n - 1

call fact(n-1)

move result of fact(n - 1) to x6

Restore caller's n

Restore caller's return address

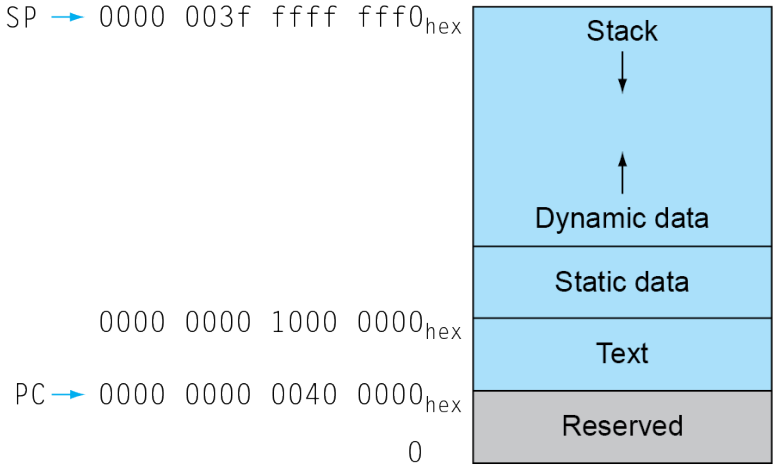
Pop stack

return n * fact(n-1)

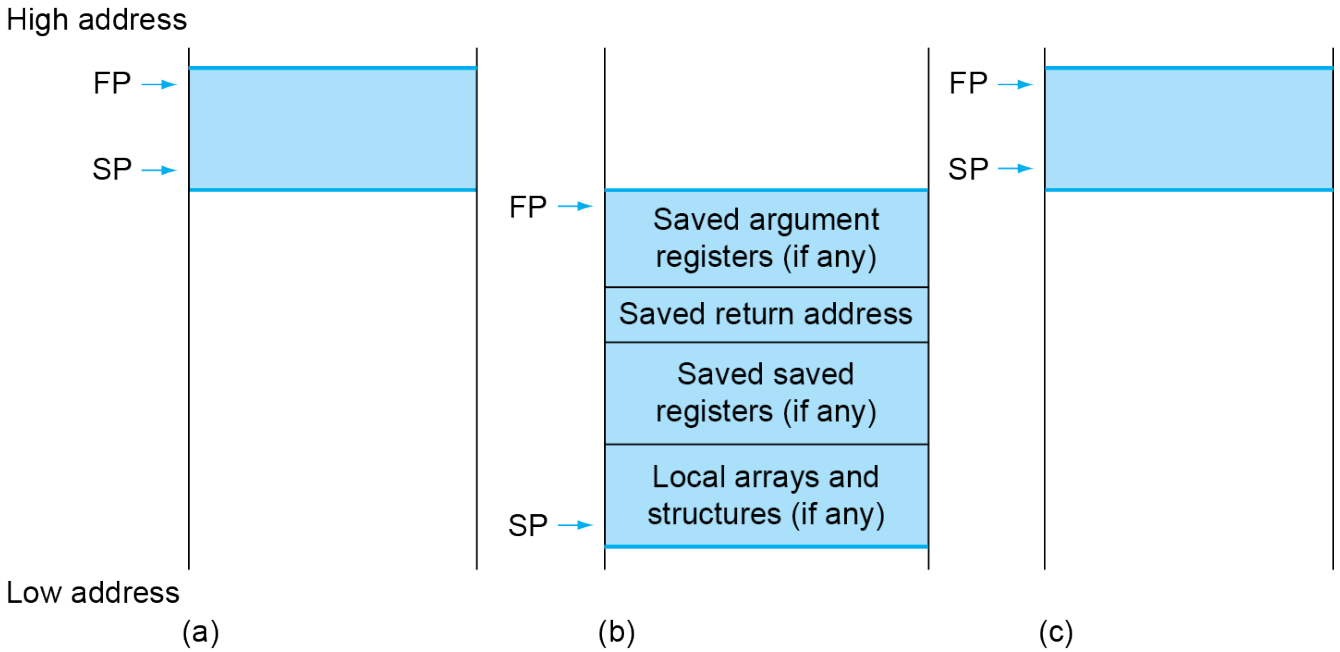
return

2.8 Memory Layout

- Text: program code
- Static data: global variables
  - e.g., static variables in C, constant arrays and strings
  - x3 (global pointer) initialized to address allowing ±offsets into this segment
- Dynamic data: heap
  - E.g., malloc in C, new in Java
- Stack: automatic storage



2.8 Local Data on the Stack



- Local data allocated by callee
  - e.g., C automatic variables
- Procedure frame (activation record)
  - Used by some compilers to manage stack storage

2.13 C Sort Example

- Illustrates use of assembly instructions for a C bubble sort function
- Swap procedure (leaf)

```
void swap(long long int v[],
          long long int k)
{
  long long int temp;
  temp = v[k];
  v[k] = v[k+1];
  v[k+1] = temp;
}
```

  - v in x10, k in x11, temp in x5

2.13 The Procedure Swap

```
swap:
  slli x6,x11,3    // reg x6 = k * 8
  add  x6,x10,x6   // reg x6 = v + (k * 8)
  ld   x5,0(x6)    // reg x5 (temp) = v[k]
  ld   x7,8(x6)    // reg x7 = v[k + 1]
  sd   x7,0(x6)    // v[k] = reg x7
  sd   x5,8(x6)    // v[k+1] = reg x5 (temp)
  jalr x0,0(x1)    // return to calling routine
```

The Sort Procedure in C

- Non-leaf (calls swap)

```
void sort (long long int v[], size_t n)
{
  size_t i, j;
  for (i = 0; i < n; i += 1) {
    for (j = i – 1;
         j >= 0 && v[j] > v[j + 1];
         j -= 1) {
      swap(v,j);
    }
  }
}
```

  - v in x10, n in x11, i in x19, j in x20

2.13 The Outer Loop

- Skeleton of outer loop:
  - for (i = 0; i <n; i += 1) {

```
   li   x19,0          // i = 0
for1tst:
   bge  x19,x11,exit1   // go to exit1 if x19 ≥ x11 (i≥n)

   (body of outer for-loop)

   addi x19,x19,1           // i += 1
   j    for1tst       // branch to test of outer loop
exit1:
```

2.13 The Inner Loop

- Skeleton of inner loop:
  - for (j = i − 1; j >= 0 && v[j] > v[j + 1]; j − = 1) {

```
        addi x20,x19,-1    // j = i −1
for2tst:
    blt  x20,x0,exit2  // go to exit2 if X20 < 0 (j < 0)
    slli x5,x20,3      // reg x5 = j * 8
    add  x5,x10,x5     // reg x5 = v + (j * 8)
    ld   x6,0(x5)      // reg x6 = v[j]
    ld   x7,8(x5)      // reg x7 = v[j + 1]
    ble  x6,x7,exit2   // go to exit2 if x6 ≤ x7
    mv   x21, x10      // copy parameter x10 into x21
    mv   x22, x11      // copy parameter x11 into x22
    mv   x10, x21      // first swap parameter is v
    mv   x11, x20      // second swap parameter is j
         jal  x1,swap       // call swap
         addi x20,x20,-1    // j −= 1
         j    for2tst       // branch to test of inner loop
 exit2:
```

2.13 Preserving Registers

- Preserve saved registers:

```
  addi sp,sp,-40  // make room on stack for 5 regs
sd   x1,32(sp)  // save x1 on stack
sd   x22,24(sp) // save x22 on stack
sd   x21,16(sp) // save x21 on stack
sd   x20,8(sp)  // save x20 on stack
sd   x19,0(sp)  // save x19 on stack
```

- Restore saved registers:

```
exit1:
  sd   x19,0(sp)     // restore x19 from stack
   sd   x20,8(sp)  // restore x20 from stack
   sd   x21,16(sp) // restore x21 from stack
   sd   x22,24(sp) // restore x22 from stack
   sd   x1,32(sp)  // restore x1 from stack
   addi sp,sp, 40  // restore stack pointer
  jalr x0,0(x1)
```